ExaminationSystem – Complete Database & Stored Procedures Documentation

Version: 1.0.0 **Author:** ExaminationSystem Team **Date:** 2025-12-31 **Environment:** Microsoft SQL Server (T-SQL)

Project Team

Basel Ashraf

Abdelaziz Gamal

Yossef Ahmed Kassab

Salah Aymn

Mahmoud Eid

Table of Contents

Executive Summary

Purpose: ExaminationSystem is a relational database designed to manage multiple training branches, tracks, courses, instructors, students, exams, questions, and grading workflows for a training center. The system supports course enrollment, supervisor relationships, creation of randomized exams from a question bank, student submissions, grading, and reporting.

Who uses it:

- **Administrators** — manage branches, tracks, courses, instructors, and global data.
- **Instructors** — assign courses, create/edit questions and choices, generate exams, view student attempts and grades.
- **Students** — enroll in courses, take exams, view their grades.

Problems solved:

- Centralized course and exam management across multiple branches/tracks.
- Controlled exam generation from per-course question banks.
- Safe submission and automated grading flow.
- Essential reports (student grades, instructor load, course topics, exam details).

High-level features:

- Branch / Track M:N mapping (tracks offered at many branches).
- Instructor supervision of specific tracks; instructors can teach many courses.
- Course-topic mapping.
- Composite-keyed Questions and Choices (course-scoped question bank).
- Exam generation (random pick of MCQ & TFs).
- Student exam attempts recording and automatic correction.
- Reporting stored procedures for common queries.

System Overview

Conceptual explanation: The system models training centers with multiple physical **Branches**. A **Track** is a program (e.g., “Data Management”) that can be offered at multiple branches (via `Branch_Track`) and typically has a supervisor (an `Instructor`). `Course` entities represent teaching modules that belong to one or more tracks (`Track_Course`). `Instructor_Course` associates instructors who teach courses. `Student` entities enroll in `Course` through `Student_Course`. `Exam` entities are tied to a course and contain `Exam_Question` rows linking to `Question` objects. `Student_Exam` tracks student attempts (registration/take), and `Student_Exam_Answer` stores answers given by a student for each question in an exam.

How pieces interact (high-level):

- Admin creates `Branch`, `Track`, `Course`, `Instructor`, `Topic`, and populates `Question` and `Choice` banks.
- Instructor (or system) runs `ExamGeneration` to create an `Exam` (random picks from `Question`).
- Students assigned to a course/track are recorded in `Student_Course`. Students are attached to an `Exam` via `Student_Exam` (indicating they will take or have taken it).
- Student submits answers via `ExamAnswer` procedure; answers are stored in `Student_Exam_Answer`.
- `ExamCorrection` grades answers, updates `Student_Exam.total_grade`, and reports results.

High-level workflow (student enrollment → exam → grading):

1. Admin/Instructor creates course, topics and question bank (MCQ & TF).
2. Students enroll in course (Student_Course).
3. Instructor generates exam (ExamGeneration).
4. Admin/Instructor registers students to the exam (insert into Student_Exam) or system does this when exam is available.
5. Student takes exam — submits answers (ExamAnswer). Answers stored in Student_Exam_Answer.
6. Instructor or automated procedure runs ExamCorrection; grades are computed and stored in Student_Exam.total_grade.
7. Reports produced for instructors, admins, and students.

Entity Relationship Diagram (ERD)

ERD Key Notes

- Primary Keys (PK) are underlined.
- Foreign Keys (FK) referenced via arrows.
- Composite Keys: Question and Choice are weak/child entities tied to Course.

- Junction tables: Branch_Track, Track_Course, Instructor_Course, Student_Course, Exam_Question, Student_Exam — used for M:N relationships.

Mermaid ERD

```
erDiagram
    BRANCH ||--o{ BRANCH_TRACK : offers
    TRACK  ||--o{ BRANCH_TRACK : offered_at
    TRACK ||--o{ TRACK_COURSE : contains
    COURSE ||--o{ TRACK_COURSE : part_of
    INSTRUCTOR ||--o{ INSTRUCTOR_COURSE : teaches
    COURSE ||--o{ INSTRUCTOR_COURSE : taught_by
    COURSE ||--o{ TOPIC : has
    TRACK ||--o{ INSTRUCTOR : supervised_by
    TRACK ||--o{ STUDENT : has_students
    STUDENT ||--o{ STUDENT_COURSE : enrolled_in
    COURSE ||--o{ STUDENT_COURSE : has_students
    COURSE ||--o{ QUESTION : owns
    QUESTION ||--o{ CHOICE : has
    COURSE ||--o{ EXAM : creates
    EXAM ||--o{ EXAM_QUESTION : contains
    QUESTION ||--o{ EXAM_QUESTION : used_in
    STUDENT ||--o{ STUDENT_EXAM : takes
    EXAM ||--o{ STUDENT_EXAM : taken_by
    STUDENT_EXAM ||--o{ STUDENT_EXAM_ANSWER : has_answers
    QUESTION ||--o{ STUDENT_EXAM_ANSWER : answered_for
    CHOICE ||--o{ STUDENT_EXAM_ANSWER : referenced_by

    BRANCH {
        int branch_id PK
        varchar branch_name
        varchar branch_location
    }
    TRACK {
        int track_id PK
        varchar track_name
        int supervisor_id FK -> INSTRUCTOR.ins_id (nullable)
    }
    BRANCH_TRACK {
        int branch_id FK
        int track_id FK
        PK(branch_id, track_id)
    }
    INSTRUCTOR {
        int ins_id PK
        varchar ins_fname
        varchar ins_lname
        char gender
        int age
        int track_id FK (nullable)
    }
    COURSE {
```

```
    int course_id PK
    varchar course_name
    int hours
}
TRACK_COURSE {
    int track_id FK
    int course_id FK
}
TOPIC {
    int topic_id PK
    varchar topic_name
    int course_id FK
}
STUDENT {
    int student_id PK
    varchar student_fname
    varchar student_lname
    int student_age
    char student_gender
    int track_id FK
}
STUDENT_COURSE {
    int student_id FK
    int course_id FK
    int grade
    PK(student_id, course_id)
}
QUESTION {
    int course_id FK
    int question_id PK (composite with course_id)
    varchar type
    varchar description
    PK(course_id, question_id)
}
CHOICE {
    int course_id FK
    int question_id FK
    int choice_id PK (composite)
    varchar choice_text
    bit is_correct
    PK(course_id, question_id, choice_id)
}
EXAM {
    int exam_id PK
    date exam_date
    int duration
    int course_id FK
}
EXAM_QUESTION {
    int exam_id FK
    int course_id FK
    int question_id FK
    PK(exam_id, course_id, question_id)
```

```
    }
    STUDENT_EXAM {
        int student_id FK
        int exam_id FK
        int total_grade
        PK(student_id, exam_id)
    }
    STUDENT_EXAM_ANSWER {
        int student_id FK
        int exam_id FK
        int course_id FK
        int question_id FK
        varchar stud_answer  -- stores choice_id as string or choice text (we document and as
        bit is_correct
        PK(student_id, exam_id, course_id, question_id)
    }
```

PlantUML ERD (downloadable snippet)

```
@startuml ExaminationSystem_ERD
entity "Branch" as Branch {
  *branch_id : INT <<PK>>
  *branch_name : VARCHAR(50)
  *branch_location : VARCHAR(50)
}
entity "Track" as Track {
  *track_id : INT <<PK>>
  *track_name : VARCHAR(50)
  supervisor_id : INT <<FK?>>
}
entity "Branch_Track" as Branch_Track {
  *branch_id : INT <<PK,FK>>
  *track_id : INT <<PK,FK>>
}
entity "Instructor" as Instructor {
  *ins_id : INT <<PK>>
  ins_fname : VARCHAR(50)
  ins_lname : VARCHAR(50)
  gender : CHAR(1)
  age : INT
  track_id : INT <<FK?>>
}
entity "Course" as Course {
  *course_id : INT <<PK>>
  course_name : VARCHAR(50)
  hours : INT
}
entity "Track_Course" as Track_Course {
  *track_id : INT <<PK,FK>>
  *course_id : INT <<PK,FK>>
}
entity "Instructor_Course" as Instructor_Course {
```

```
  *ins_id : INT <<PK,FK>>
  *course_id : INT <<PK,FK>>
}
entity "Topic" as Topic {
  *topic_id : INT <<PK>>
  topic_name : VARCHAR(50)
  course_id : INT <<FK>>
}
entity "Student" as Student {
  *student_id : INT <<PK>>
  student_fname : VARCHAR(50)
  student_lname : VARCHAR(50)
  student_age : INT
  student_gender : CHAR(1)
  track_id : INT <<FK>>
}
entity "Student_Course" as Student_Course {
  *student_id : INT <<PK,FK>>
  *course_id : INT <<PK,FK>>
  grade : INT
}
entity "Question" as Question {
  *course_id : INT <<PK,FK>>
  *question_id : INT <<PK>>
  type : VARCHAR(10)
  description : VARCHAR(200)
}
entity "Choice" as Choice {
  *course_id : INT <<PK,FK>>
  *question_id : INT <<PK,FK>>
  *choice_id : INT <<PK>>
  choice_text : VARCHAR(150)
  is_correct : BIT
}
entity "Exam" as Exam {
  *exam_id : INT <<PK>>
  exam_date : DATE
  duration : INT
  course_id : INT <<FK>>
}
entity "Exam_Question" as Exam_Question {
  *exam_id : INT <<PK,FK>>
  *course_id : INT <<PK,FK>>
  *question_id : INT <<PK,FK>>
}
entity "Student_Exam" as Student_Exam {
  *student_id : INT <<PK,FK>>
  *exam_id : INT <<PK,FK>>
  total_grade : INT
}
entity "Student_Exam_Answer" as Student_Exam_Answer {
  *student_id : INT <<PK,FK>>
  *exam_id : INT <<PK,FK>>
```

```
  *course_id : INT <<PK,FK>>
  *question_id : INT <<PK,FK>>
  stud_answer : VARCHAR(255)
  is_correct : BIT
}

Branch ||--o{ Branch_Track
Track ||--o{ Branch_Track
Track ||--o{ Track_Course
Course ||--o{ Track_Course
Instructor ||--o{ Instructor_Course
Course ||--o{ Instructor_Course
Course ||--o{ Topic
Track ||--o{ Instructor
Track ||--o{ Student
Student ||--o{ Student_Course
Course ||--o{ Student_Course
Course ||--o{ Question
Question ||--o{ Choice
Course ||--o{ Exam
Exam ||--o{ Exam_Question
Question ||--o{ Exam_Question
Student ||--o{ Student_Exam
Exam ||--o{ Student_Exam
Student_Exam ||--o{ Student_Exam_Answer
Question ||--o{ Student_Exam_Answer
Choice ||--o{ Student_Exam_Answer

@enduml
```

Database Schema (Full Detail)

All statements below are **T-SQL (SQL Server)** and are idempotent where appropriate (use `IF NOT EXISTS` guards). These reflect the current schema and constraints used in the project.

Assumption (explicit): `Student_Exam_Answer.stud_answer` stores the student's selected `choice_id` as text (string) because choice ids are integers but earlier code used `VARCHAR`. Implementation ensures comparisons cast accordingly. This is documented consistently later.

Conventions used:

- `INT` primary keys unless specified.
- `BIT` for boolean values.
- `VARCHAR` lengths chosen to match earlier schema.
- Composite PKs kept for `Question` and `Choice`.

---

1. Table: Branch

Purpose: Stores physical training branches.

Columns:

- `branch_id` INT NOT NULL PRIMARY KEY
- `branch_name` VARCHAR(50) NOT NULL
- `branch_location` VARCHAR(50) NOT NULL

SQL:

```
IF OBJECT_ID('dbo.Branch') IS NULL
BEGIN
CREATE TABLE dbo.Branch (
    branch_id INT NOT NULL PRIMARY KEY,
    branch_name VARCHAR(50) NOT NULL,
    branch_location VARCHAR(50) NOT NULL
);
END;
```

---

2. Table: Track

Purpose: Training program or track (e.g., Data Management). Tracks can be offered at many branches.

Columns:

- `track_id` INT NOT NULL PRIMARY KEY
- `track_name` VARCHAR(50) NOT NULL
- `supervisor_id` INT NULL — FK to `Instructor(ins_id)` ; unique when not NULL (a track supervisor is an instructor who belongs to that track).

Business meaning: A track may have a designated supervising instructor (`supervisor_id`) who ideally is assigned a `track_id` matching this track. The project enforces supervisor-instructor relation via triggers.

SQL:

```
IF OBJECT_ID('dbo.Track') IS NULL
BEGIN
CREATE TABLE dbo.Track (
    track_id INT NOT NULL PRIMARY KEY,
    track_name VARCHAR(50) NOT NULL,
    supervisor_id INT NULL
);
-- Unique index to prevent multiple tracks pointing to same supervisor (if that is desired)
CREATE UNIQUE INDEX UX_Track_Supervisor
ON dbo.Track(supervisor_id)
```

```
WHERE supervisor_id IS NOT NULL;
END;
```

---

3. Table: Branch_Track (junction)

Purpose: M:N mapping of which tracks are offered at which branches.

Columns & PK:

- `branch_id` INT NOT NULL — FK → Branch(branch_id)
- `track_id` INT NOT NULL — FK → Track(track_id)
- Primary Key: (`branch_id` , `track_id`)

SQL:

```
IF OBJECT_ID('dbo.Branch_Track') IS NULL
BEGIN
CREATE TABLE dbo.Branch_Track (
    branch_id INT NOT NULL,
    track_id INT NOT NULL,
    CONSTRAINT PK_Branch_Track PRIMARY KEY (branch_id, track_id),
    CONSTRAINT FK_BranchTrack_Branch FOREIGN KEY (branch_id) REFERENCES dbo.Branch(branch_id)
    CONSTRAINT FK_BranchTrack_Track  FOREIGN KEY (track_id)  REFERENCES dbo.Track(track_id)
);
END;
```

---

4. Table: Instructor

Purpose: Instructor records.

Columns:

- `ins_id` INT NOT NULL PRIMARY KEY
- `ins_fname` VARCHAR(50) NOT NULL
- `ins_lname` VARCHAR(50) NOT NULL
- `gender` CHAR(1) NULL CHECK IN ('M','F')
- `age` INT NULL CHECK (age > 20)
- `track_id` INT NULL — FK to Track(track_id). Instructor may be unassigned.

Business meaning: An instructor has an assigned track (may be NULL). A track may reference a supervisor via `supervisor_id` .

SQL:

```
IF OBJECT_ID('dbo.Instructor') IS NULL
BEGIN
CREATE TABLE dbo.Instructor (
```

```
    ins_id INT NOT NULL PRIMARY KEY,
    ins_fname VARCHAR(50) NOT NULL,
    ins_lname VARCHAR(50) NOT NULL,
    gender CHAR(1) NULL CHECK (gender IN ('M','F')),
    age INT NULL CHECK (age > 20),
    track_id INT NULL,
    CONSTRAINT FK_Instructor_Track FOREIGN KEY (track_id) REFERENCES dbo.Track(track_id)
);
END;
-- Add FK from Track.supervisor_id to Instructor.ins_id (after Instructor exists)
IF NOT EXISTS (
    SELECT 1 FROM sys.foreign_keys WHERE name = 'FK_Track_Supervisor'
)
BEGIN
ALTER TABLE dbo.Track
ADD CONSTRAINT FK_Track_Supervisor
FOREIGN KEY (supervisor_id) REFERENCES dbo.Instructor(ins_id);
END;
```

---

5. Table: Course

Purpose: Courses (class modules).

Columns:

- `course_id` INT NOT NULL PRIMARY KEY
- `course_name` VARCHAR(50) NOT NULL
- `hours` INT NOT NULL CHECK (hours > 0)

SQL:

```
IF OBJECT_ID('dbo.Course') IS NULL
BEGIN
CREATE TABLE dbo.Course (
    course_id INT NOT NULL PRIMARY KEY,
    course_name VARCHAR(50) NOT NULL,
    hours INT NOT NULL CHECK (hours > 0)
);
END;
```

---

6. Table: Track_Course (junction)

Purpose: M:N mapping of track to courses.

Columns:

- `track_id` INT NOT NULL — FK → Track(track_id)
- `course_id` INT NOT NULL — FK → Course(course_id)

- PK (`track_id` , `course_id`)

SQL:

```
IF OBJECT_ID('dbo.Track_Course') IS NULL
BEGIN
CREATE TABLE dbo.Track_Course (
    track_id INT NOT NULL,
    course_id INT NOT NULL,
    CONSTRAINT PK_Track_Course PRIMARY KEY (track_id, course_id),
    CONSTRAINT FK_TrackCourse_Track FOREIGN KEY (track_id) REFERENCES dbo.Track(track_id),
    CONSTRAINT FK_TrackCourse_Course FOREIGN KEY (course_id) REFERENCES dbo.Course(course_id)
);
END;
```

---

7. Table: Instructor_Course (junction)

Purpose: Which instructors teach which courses (M:N).

Columns:

- `ins_id` INT NOT NULL — FK → Instructor(ins_id)
- `course_id` INT NOT NULL — FK → Course(course_id)
- PK (`ins_id` , `course_id`)

SQL:

```
IF OBJECT_ID('dbo.Instructor_Course') IS NULL
BEGIN
CREATE TABLE dbo.Instructor_Course (
    ins_id INT NOT NULL,
    course_id INT NOT NULL,
    CONSTRAINT PK_Instructor_Course PRIMARY KEY (ins_id, course_id),
    CONSTRAINT FK_InstructorCourse_Instructor FOREIGN KEY (ins_id) REFERENCES dbo.Instructor(
    CONSTRAINT FK_InstructorCourse_Course FOREIGN KEY (course_id) REFERENCES dbo.Course(cours
);
END;
```

---

8. Table: Topic

Purpose: Sub-topics within courses (e.g., HTML, CSS).

Columns:

- `topic_id` INT NOT NULL PRIMARY KEY
- `topic_name` VARCHAR(50) NOT NULL
- `course_id` INT NOT NULL — FK → Course(course_id)

SQL:

```
IF OBJECT_ID('dbo.Topic') IS NULL
BEGIN
CREATE TABLE dbo.Topic (
    topic_id INT NOT NULL PRIMARY KEY,
    topic_name VARCHAR(50) NOT NULL,
    course_id INT NOT NULL,
    CONSTRAINT FK_Topic_Course FOREIGN KEY (course_id) REFERENCES dbo.Course(course_id)
);
END;
```

---

9. Table: Student

Purpose: Student records.

Columns:

- `student_id` INT NOT NULL PRIMARY KEY
- `student_fname` VARCHAR(50) NOT NULL
- `student_lname` VARCHAR(50) NOT NULL
- `student_age` INT NOT NULL CHECK (student_age >= 18)
- `student_gender` CHAR(1) NOT NULL CHECK (student_gender IN ('M','F'))
- `track_id` INT NOT NULL — FK → Track(track_id)

SQL:

```
IF OBJECT_ID('dbo.Student') IS NULL
BEGIN
CREATE TABLE dbo.Student (
    student_id INT NOT NULL PRIMARY KEY,
    student_fname VARCHAR(50) NOT NULL,
    student_lname VARCHAR(50) NOT NULL,
    student_age INT NOT NULL CHECK (student_age >= 18),
    student_gender CHAR(1) NOT NULL CHECK (student_gender IN ('M','F')),
    track_id INT NOT NULL,
    CONSTRAINT FK_Student_Track FOREIGN KEY (track_id) REFERENCES dbo.Track(track_id)
);
END;
```

---

10. Table: Student_Course (junction)

Purpose: Enrollment (student taking a course). Contains grade (0..100).

Columns:

- `student_id` INT NOT NULL — FK → Student(student_id)

- `course_id` INT NOT NULL — FK → Course(course_id)
- `grade` INT NULL CHECK (grade BETWEEN 0 AND 100)
- PK (`student_id` , `course_id`)

SQL:

```
IF OBJECT_ID('dbo.Student_Course') IS NULL
BEGIN
CREATE TABLE dbo.Student_Course (
    student_id INT NOT NULL,
    course_id INT NOT NULL,
    grade INT NULL CHECK (grade BETWEEN 0 AND 100),
    CONSTRAINT PK_Student_Course PRIMARY KEY (student_id, course_id),
    CONSTRAINT FK_StudentCourse_Student FOREIGN KEY (student_id) REFERENCES dbo.Student(stude
    CONSTRAINT FK_StudentCourse_Course  FOREIGN KEY (course_id)  REFERENCES dbo.Course(course
);
END;
```

---

11. Table: Question (weak / composite PK)

Purpose: Questions belong to a specific course. Composite PK (course_id + question_id) — a `question_id` is unique only per course.

Columns:

- `course_id` INT NOT NULL — FK → Course(course_id)
- `question_id` INT NOT NULL — (composite PK)
- `type` VARCHAR(10) NOT NULL CHECK IN ('MCQ','TF')
- `description` VARCHAR(200) NOT NULL
- PK (`course_id` , `question_id`)

SQL:

```
IF OBJECT_ID('dbo.Question') IS NULL
BEGIN
CREATE TABLE dbo.Question (
    course_id INT NOT NULL,
    question_id INT NOT NULL,
    type VARCHAR(10) NOT NULL CHECK (type IN ('MCQ','TF')),
    description VARCHAR(200) NOT NULL,
    CONSTRAINT PK_Question PRIMARY KEY (course_id, question_id),
    CONSTRAINT FK_Question_Course FOREIGN KEY (course_id) REFERENCES dbo.Course(course_id)
);
END;
```

---

12. Table: Choice (weak / composite PK)

Purpose: Choices for MCQs / TF questions. Composite PK (`course_id` , `question_id` , `choice_id`).

Columns:

- `course_id` INT NOT NULL — FK → Question(course_id)
- `question_id` INT NOT NULL — FK → Question(question_id)
- `choice_id` INT NOT NULL — choice number within question (1..N)
- `choice_text` VARCHAR(150) NOT NULL
- `is_correct` BIT NOT NULL
- PK (`course_id` , `question_id` , `choice_id`)

SQL:

```
IF OBJECT_ID('dbo.Choice') IS NULL
BEGIN
CREATE TABLE dbo.Choice (
    course_id INT NOT NULL,
    question_id INT NOT NULL,
    choice_id INT NOT NULL,
    choice_text VARCHAR(150) NOT NULL,
    is_correct BIT NOT NULL,
    CONSTRAINT PK_Choice PRIMARY KEY (course_id, question_id, choice_id),
    CONSTRAINT FK_Choice_Question FOREIGN KEY (course_id, question_id)
        REFERENCES dbo.Question(course_id, question_id)
);
END;
```

---

13. Table: Exam

Purpose: An exam is tied to a course and has a date/duration.

Columns:

- `exam_id` INT NOT NULL PRIMARY KEY
- `exam_date` DATE NOT NULL
- `duration` INT NOT NULL — length in minutes
- `course_id` INT NOT NULL — FK → Course(course_id)

SQL:

```
IF OBJECT_ID('dbo.Exam') IS NULL
BEGIN
CREATE TABLE dbo.Exam (
    exam_id INT NOT NULL PRIMARY KEY,
    exam_date DATE NOT NULL,
    duration INT NOT NULL CHECK (duration > 0),
    course_id INT NOT NULL,
    CONSTRAINT FK_Exam_Course FOREIGN KEY (course_id) REFERENCES dbo.Course(course_id)
```

```
);
END;
```

---

14. Table: Exam_Question (junction)

Purpose: Which questions belong to an exam. Composite PK (`exam_id` , `course_id` , `question_id`) matching the course scope.

Columns:

- `exam_id` INT NOT NULL — FK → Exam(exam_id)
- `course_id` INT NOT NULL — FK → Question(course_id)
- `question_id` INT NOT NULL — FK → Question(question_id)
- PK (`exam_id` , `course_id` , `question_id`)

SQL:

```
IF OBJECT_ID('dbo.Exam_Question') IS NULL
BEGIN
CREATE TABLE dbo.Exam_Question (
    exam_id INT NOT NULL,
    course_id INT NOT NULL,
    question_id INT NOT NULL,
    CONSTRAINT PK_Exam_Question PRIMARY KEY (exam_id, course_id, question_id),
    CONSTRAINT FK_ExamQuestion_Exam FOREIGN KEY (exam_id) REFERENCES dbo.Exam(exam_id),
    CONSTRAINT FK_ExamQuestion_Question FOREIGN KEY (course_id, question_id)
        REFERENCES dbo.Question(course_id, question_id)
);
END;
```

> **Note:** As discussed in system design, persistent question order is not stored (no `question_order` column). The system shuffles at **read time** (recommended) — see "Exam Workflow" and "ExamGeneration" details.

---

15. Table: Student_Exam (junction / attempt registration)

Purpose: Record that a student is associated with an exam (attempt). `total_grade` stores final grade as integer percent (0..100) or NULL until graded.

Columns:

- `student_id` INT NOT NULL — FK → Student(student_id)
- `exam_id` INT NOT NULL — FK → Exam(exam_id)
- `total_grade` INT NULL CHECK (total_grade BETWEEN 0 AND 100)
- PK (`student_id` , `exam_id`)

SQL:

```
IF OBJECT_ID('dbo.Student_Exam') IS NULL
BEGIN
CREATE TABLE dbo.Student_Exam (
    student_id INT NOT NULL,
    exam_id INT NOT NULL,
    total_grade INT NULL CHECK (total_grade BETWEEN 0 AND 100),
    CONSTRAINT PK_Student_Exam PRIMARY KEY (student_id, exam_id),
    CONSTRAINT FK_StudentExam_Student FOREIGN KEY (student_id) REFERENCES dbo.Student(student
    CONSTRAINT FK_StudentExam_Exam FOREIGN KEY (exam_id) REFERENCES dbo.Exam(exam_id)
);
END;
```

---

16. Table: Student_Exam_Answer (dependent / ternary junction)

Purpose: Stores a student's answer for each question in an exam. Composite PK (`student_id` , `exam_id` , `course_id` , `question_id`). `stud_answer` stores the `choice_id` as text (or the textual answer in the case of open answers — current design uses choice ids for MCQ/TF).

Columns:

- `student_id` INT NOT NULL — FK → Student(student_id)
- `exam_id` INT NOT NULL — FK → Exam(exam_id)
- `question_id` INT NOT NULL — FK → Question(question_id)
- `course_id` INT NOT NULL — FK → Question(course_id)
- `stud_answer` VARCHAR(255) NULL — typically stores choice_id as string (e.g., '1','2') or textual answers
- `is_correct` BIT NULL — 1 if correct, 0 if incorrect (can be updated after correction)
- PK (`student_id` , `exam_id` , `course_id` , `question_id`)

SQL:

```
IF OBJECT_ID('dbo.Student_Exam_Answer') IS NULL
BEGIN
CREATE TABLE dbo.Student_Exam_Answer (
    student_id INT NOT NULL,
    exam_id INT NOT NULL,
    question_id INT NOT NULL,
    course_id INT NOT NULL,
    stud_answer VARCHAR(255) NULL,
    is_correct BIT NULL,
    CONSTRAINT PK_Student_Exam_Answer PRIMARY KEY (student_id, exam_id, course_id, question_i
    CONSTRAINT FK_StudentExamAnswer_StudentExam FOREIGN KEY (student_id, exam_id)
        REFERENCES dbo.Student_Exam(student_id, exam_id),
    CONSTRAINT FK_StudentExamAnswer_Question FOREIGN KEY (course_id, question_id)
        REFERENCES dbo.Question(course_id, question_id)
);
END;
```

Design note: This is effectively a quaternary relation keyed by student, exam, course, question. It is required because answers are specific to a student's attempt to a given exam for a course. This design is intentionally normalized to avoid duplication.

Business Rules & Constraints

This section enumerates critical business rules enforced either by schema constraints, triggers, or application logic (documented SPs enforce them). Each rule includes rationale and enforcement mechanism.

1. **Branches & Tracks**
    - A `Track` can be offered at multiple `Branch` via `Branch_Track`. (Enforced via the junction table).
    - A `Track` may optionally have a `supervisor_id`. If set, the referenced Instructor must exist. (FK enforced).
2. **Instructor ↔ Track**
    - An `Instructor` may be assigned to one track (`Instructor.track_id`).
    - `Track.supervisor_id` references an `Instructor`. Business rule: the supervisor instructor should belong to that track (enforced by trigger `trg_track_supervisor_same_track` in practice).
    - The unique index `UX_Track_Supervisor` ensures a given instructor cannot be supervisor of multiple tracks unless `NULL` logic adjusted.
3. **Courses & Topics**
    - `Topic` rows map to `Course`. Each course may have many topics. Topics help reporting.
4. **Questions & Choices**
    - Each `Question` belongs to a single `Course`. PK is composite `(course_id, question_id)` — meaning `question_id` is only unique per course.
    - `Choice` rows are scoped by `(course_id, question_id)` and each `choice_id` is per-question.
    - For MCQ questions, one (and only one) `Choice.is_correct = 1` must exist. This is enforced via procedures and data validation (no schema CHECK can easily enforce "exactly one true" per question).
    - For TF questions, choices are usually `1:'True'` and `2:'False'` with one marked correct.
5. **Student Enrollment & Eligibility**
    - A `Student` may enroll in many `Course` s via `Student_Course`.
    - A student must be enrolled (`Student_Course`) in a course to be eligible to take the course's `Exam`. `ExamAnswer` procedure checks this.

6. **Exam Generation**
    - Exams are generated using `ExamGeneration` stored procedure, which picks N MCQ and M TF questions from the course question bank. Inserted order is not persistent; reading clients must `ORDER BY NEWID()` or use deterministic ordering if needed.
7. **Exam Attempts & Answer Submission**
    - `Student_Exam` must exist for a given (`student_id` , `exam_id`) before student may submit answers (this ensures the student is attached to the exam). `ExamAnswer` validates presence.
    - Students cannot attempt the same exam twice. `ExamAnswer` and also `StudentExam_Insert` enforce duplicate prevention by checking `Student_Exam` existence and whether `total_grade` is NULL or attempt entries exist.
8. **Answer & Correction Rules**
    - `Student_Exam_Answer.stud_answer` stores the choice_id (as string) for MCQ/TF. `ExamCorrection` maps `stud_answer` to the `Choice` where `is_correct = 1` . `ExamCorrection` updates `is_correct` (1/0) and computes `Student_Exam.total_grade` as integer percent.
    - Trigger `trg_answer_correctness` ensures `is_correct` values inserted are only 0/1.
9. **Referential Integrity**
    - All specified FKs are in place. Insert ordering for seeds must respect FKs: Branch → Track → Instructor → Course → Question → Choice → Exam → Exam_Question → Student → Student_Course → Student_Exam → Student_Exam_Answer.
10. **Normalization**
    - The database is in 3NF: no repeating groups, composite keys for weak entities, no transitive dependencies in column definitions.

Stored Procedures — Full Documentation

Format per procedure: Purpose → Parameters → Behavior → Validation → Error handling → Example calls → Expected results → Edge cases.

All stored procedures are provided as `CREATE OR ALTER PROCEDURE` blocks to be idempotent.

Important: For long sections, code blocks are labeled with `sql` .

CRUD Procedures (Branch)

Branch_Insert

Purpose: Insert a branch.

Parameters:

- `@id INT`
- `@name VARCHAR(50)`
- `@loc VARCHAR(50)`

Behavior: Inserts row into `Branch`. Fails if PK exists.

SQL:

```
CREATE OR ALTER PROCEDURE Branch_Insert
    @id INT, @name VARCHAR(50), @loc VARCHAR(50)
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (SELECT 1 FROM dbo.Branch WHERE branch_id = @id)
    BEGIN
        RAISERROR('Branch id already exists',16,1); RETURN;
    END
    INSERT INTO dbo.Branch(branch_id, branch_name, branch_location)
    VALUES(@id, @name, @loc);
END;
GO
```

Example:

```
EXEC Branch_Insert @id=10, @name='Ismailia', @loc='Ismailia City';
```

Edge cases: Name too long -> SQL error; duplicate id -> raised.

Branch_GetAll

Purpose: Select all branches.

```
CREATE OR ALTER PROCEDURE Branch_GetAll
AS
BEGIN
    SELECT branch_id, branch_name, branch_location FROM dbo.Branch ORDER BY branch_name;
END;
GO
```

Branch_GetById

Purpose: Get branch by id.

```
CREATE OR ALTER PROCEDURE Branch_GetById (@id INT)
AS
```

```
BEGIN
    SELECT branch_id, branch_name, branch_location FROM dbo.Branch WHERE branch_id=@id;
END;
GO
```

Branch_Update

Purpose: Update branch.

```
CREATE OR ALTER PROCEDURE Branch_Update (@id INT,@name VARCHAR(50),@loc VARCHAR(50))
AS
BEGIN
    IF NOT EXISTS (SELECT 1 FROM dbo.Branch WHERE branch_id=@id)
    BEGIN
        RAISERROR('Branch not found',16,1); RETURN;
    END
    UPDATE dbo.Branch SET branch_name=@name, branch_location=@loc WHERE branch_id=@id;
END;
GO
```

Branch_Delete

Purpose: Delete branch (ensure no dependent rows in Branch_Track etc.)

```
CREATE OR ALTER PROCEDURE Branch_Delete (@id INT)
AS
BEGIN
    IF EXISTS (SELECT 1 FROM dbo.Branch_Track WHERE branch_id=@id)
    BEGIN
        RAISERROR('Cannot delete branch: dependent tracks exist',16,1); RETURN;
    END
    DELETE FROM dbo.Branch WHERE branch_id=@id;
END;
GO
```

> The same pattern applies to Course, Track, Topic, Instructor, Student, Question, Choice, Exam, and the junction tables. Below we provide all the CRUD SPs (compactly but fully). Each SP validates existence and referential integrity and raises clear errors.

CRUD Procedures (Course)

```
CREATE OR ALTER PROCEDURE Course_Insert (@id INT,@name VARCHAR(50),@hours INT)
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS(SELECT 1 FROM dbo.Course WHERE course_id=@id) BEGIN RAISERROR('Course exists',1
    INSERT INTO dbo.Course(course_id, course_name, hours) VALUES(@id,@name,@hours);
```

```
END;
GO

CREATE OR ALTER PROCEDURE Course_GetAll AS
BEGIN
    SELECT course_id, course_name, hours FROM dbo.Course ORDER BY course_name;
END;
GO

CREATE OR ALTER PROCEDURE Course_GetById (@id INT) AS
BEGIN
    SELECT * FROM dbo.Course WHERE course_id=@id;
END;
GO

CREATE OR ALTER PROCEDURE Course_Update (@id INT,@name VARCHAR(50),@hours INT) AS
BEGIN
    IF NOT EXISTS(SELECT 1 FROM dbo.Course WHERE course_id=@id) BEGIN RAISERROR('Course not f
    UPDATE dbo.Course SET course_name=@name, hours=@hours WHERE course_id=@id;
END;
GO

CREATE OR ALTER PROCEDURE Course_Delete (@id INT) AS
BEGIN
    IF EXISTS(SELECT 1 FROM dbo.Track_Course WHERE course_id=@id) BEGIN RAISERROR('Cannot del
    DELETE FROM dbo.Course WHERE course_id=@id;
END;
GO
```

CRUD Procedures (Track)

```
CREATE OR ALTER PROCEDURE Track_Insert (@id INT, @name VARCHAR(50), @sup INT = NULL)
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS(SELECT 1 FROM dbo.Track WHERE track_id=@id) BEGIN RAISERROR('Track exists',16,1
    INSERT INTO dbo.Track(track_id, track_name, supervisor_id) VALUES(@id, @name, @sup);
END;
GO

CREATE OR ALTER PROCEDURE Track_GetAll AS
BEGIN
    SELECT track_id, track_name, supervisor_id FROM dbo.Track ORDER BY track_name;
END;
GO

CREATE OR ALTER PROCEDURE Track_GetById (@id INT) AS
BEGIN
    SELECT * FROM dbo.Track WHERE track_id=@id;
END;
GO
```

```
CREATE OR ALTER PROCEDURE Track_Update (@id INT, @name VARCHAR(50), @sup INT = NULL) AS
BEGIN
    IF NOT EXISTS(SELECT 1 FROM dbo.Track WHERE track_id=@id) BEGIN RAISERROR('Track not foun
    UPDATE dbo.Track SET track_name=@name, supervisor_id=@sup WHERE track_id=@id;
END;
GO

CREATE OR ALTER PROCEDURE Track_Delete (@id INT) AS
BEGIN
    IF EXISTS(SELECT 1 FROM dbo.Branch_Track WHERE track_id=@id) BEGIN RAISERROR('Cannot dele
    DELETE FROM dbo.Track WHERE track_id=@id;
END;
GO
```

CRUD Procedures (Topic)

```
CREATE OR ALTER PROCEDURE Topic_Insert (@id INT,@name VARCHAR(50),@course INT)
AS
BEGIN
    IF EXISTS(SELECT 1 FROM dbo.Topic WHERE topic_id=@id) BEGIN RAISERROR('Topic exists',16,1
    IF NOT EXISTS(SELECT 1 FROM dbo.Course WHERE course_id=@course) BEGIN RAISERROR('Course n
    INSERT INTO dbo.Topic(topic_id, topic_name, course_id) VALUES(@id,@name,@course);
END;
GO

CREATE OR ALTER PROCEDURE Topic_GetAll AS
BEGIN SELECT topic_id, topic_name, course_id FROM dbo.Topic ORDER BY topic_name; END;
GO
CREATE OR ALTER PROCEDURE Topic_GetById (@id INT) AS SELECT * FROM dbo.Topic WHERE topic_id=@
CREATE OR ALTER PROCEDURE Topic_Update (@id INT,@name VARCHAR(50),@course INT) AS
BEGIN
    IF NOT EXISTS(SELECT 1 FROM dbo.Topic WHERE topic_id=@id) BEGIN RAISERROR('Topic not foun
    UPDATE dbo.Topic SET topic_name=@name, course_id=@course WHERE topic_id=@id;
END; GO
CREATE OR ALTER PROCEDURE Topic_Delete (@id INT) AS DELETE FROM dbo.Topic WHERE topic_id=@id;
```

CRUD Procedures (Instructor)

```
CREATE OR ALTER PROCEDURE Instructor_Insert (@id INT,@fn VARCHAR(50),@ln VARCHAR(50),@g CHAR(
AS
BEGIN
    IF EXISTS(SELECT 1 FROM dbo.Instructor WHERE ins_id=@id) BEGIN RAISERROR('Instructor exis
    INSERT INTO dbo.Instructor(ins_id, ins_fname, ins_lname, gender, age, track_id) VALUES(@i
END;
GO

CREATE OR ALTER PROCEDURE Instructor_GetAll AS SELECT * FROM dbo.Instructor ORDER BY ins_lnam
```

```
CREATE OR ALTER PROCEDURE Instructor_GetById (@id INT) AS SELECT * FROM dbo.Instructor WHERE

CREATE OR ALTER PROCEDURE Instructor_Update (@id INT,@fn VARCHAR(50),@ln VARCHAR(50),@g CHAR(
BEGIN
    IF NOT EXISTS(SELECT 1 FROM dbo.Instructor WHERE ins_id=@id) BEGIN RAISERROR('Instructor
    UPDATE dbo.Instructor SET ins_fname=@fn, ins_lname=@ln, gender=@g, age=@age, track_id=@tr
END; GO

CREATE OR ALTER PROCEDURE Instructor_Delete (@id INT) AS
BEGIN
    IF EXISTS(SELECT 1 FROM dbo.Instructor_Course WHERE ins_id=@id) BEGIN RAISERROR('Cannot d
    DELETE FROM dbo.Instructor WHERE ins_id=@id;
END; GO
```

CRUD Procedures (Student)

```
CREATE OR ALTER PROCEDURE Student_Insert (@id INT,@fn VARCHAR(50),@ln VARCHAR(50),@age INT,@g
AS
BEGIN
    IF EXISTS(SELECT 1 FROM dbo.Student WHERE student_id=@id) BEGIN RAISERROR('Student exists
    INSERT INTO dbo.Student(student_id, student_fname, student_lname, student_age, student_ge
    VALUES(@id,@fn,@ln,@age,@g,@track);
END;
GO

CREATE OR ALTER PROCEDURE Student_GetAll AS SELECT * FROM dbo.Student ORDER BY student_lname,
CREATE OR ALTER PROCEDURE Student_GetById (@id INT) AS SELECT * FROM dbo.Student WHERE studen

CREATE OR ALTER PROCEDURE Student_Update (@id INT,@fn VARCHAR(50),@ln VARCHAR(50),@age INT,@g
AS
BEGIN
    IF NOT EXISTS(SELECT 1 FROM dbo.Student WHERE student_id=@id) BEGIN RAISERROR('Student no
    UPDATE dbo.Student SET student_fname=@fn, student_lname=@ln, student_age=@age, student_ge
END; GO

CREATE OR ALTER PROCEDURE Student_Delete (@id INT) AS
BEGIN
    IF EXISTS(SELECT 1 FROM dbo.Student_Course WHERE student_id=@id) BEGIN RAISERROR('Cannot
    DELETE FROM dbo.Student WHERE student_id=@id;
END; GO
```

CRUD Procedures (Question & Choice)

Important: `Question` uses composite PK; `Choice` uses composite PK.

```
CREATE OR ALTER PROCEDURE Question_Insert (@course INT,@qid INT,@type VARCHAR(10),@desc VARCH
AS
BEGIN
    IF EXISTS(SELECT 1 FROM dbo.Question WHERE course_id=@course AND question_id=@qid) BEGIN
```

```
    INSERT INTO dbo.Question(course_id, question_id, type, description) VALUES(@course,@qid,@
END;
GO

CREATE OR ALTER PROCEDURE Question_Get (@course INT,@qid INT) AS
BEGIN SELECT * FROM dbo.Question WHERE course_id=@course AND question_id=@qid; END;
GO

CREATE OR ALTER PROCEDURE Question_Update (@course INT,@qid INT,@type VARCHAR(10),@desc VARCH
BEGIN
    IF NOT EXISTS(SELECT 1 FROM dbo.Question WHERE course_id=@course AND question_id=@qid) BE
    UPDATE dbo.Question SET type=@type, description=@desc WHERE course_id=@course AND questio
END;
GO

CREATE OR ALTER PROCEDURE Question_Delete (@course INT,@qid INT) AS
BEGIN
    IF EXISTS(SELECT 1 FROM dbo.Choice WHERE course_id=@course AND question_id=@qid) BEGIN RA
    DELETE FROM dbo.Question WHERE course_id=@course AND question_id=@qid;
END;
GO

CREATE OR ALTER PROCEDURE Choice_Insert (@course INT,@qid INT,@cid INT,@text VARCHAR(150),@co
AS
BEGIN
    IF NOT EXISTS(SELECT 1 FROM dbo.Question WHERE course_id=@course AND question_id=@qid) BE
    IF EXISTS(SELECT 1 FROM dbo.Choice WHERE course_id=@course AND question_id=@qid AND choic
    INSERT INTO dbo.Choice(course_id, question_id, choice_id, choice_text, is_correct) VALUES
END;
GO

CREATE OR ALTER PROCEDURE Choice_Get (@course INT,@qid INT) AS SELECT * FROM dbo.Choice WHERE

CREATE OR ALTER PROCEDURE Choice_GetById (@course INT,@qid INT,@cid INT) AS
BEGIN SELECT * FROM dbo.Choice WHERE course_id=@course AND question_id=@qid AND choice_id=@ci
GO

CREATE OR ALTER PROCEDURE Choice_Update (@course INT,@qid INT,@cid INT,@text VARCHAR(150),@co
BEGIN
    IF NOT EXISTS(SELECT 1 FROM dbo.Choice WHERE course_id=@course AND question_id=@qid AND c
    UPDATE dbo.Choice SET choice_text=@text, is_correct=@correct WHERE course_id=@course AND
END;
GO

CREATE OR ALTER PROCEDURE Choice_Delete (@course INT,@qid INT,@cid INT) AS
BEGIN DELETE FROM dbo.Choice WHERE course_id=@course AND question_id=@qid AND choice_id=@cid;
GO
```

CRUD Procedures (Exam & Exam_Question)

```
CREATE OR ALTER PROCEDURE Exam_Insert (@id INT,@date DATE,@dur INT,@course INT)
AS
BEGIN
    IF EXISTS(SELECT 1 FROM dbo.Exam WHERE exam_id=@id) BEGIN RAISERROR('Exam exists',16,1);
    INSERT INTO dbo.Exam(exam_id, exam_date, duration, course_id) VALUES(@id,@date,@dur,@cour
END;
GO

CREATE OR ALTER PROCEDURE Exam_GetAll AS SELECT * FROM dbo.Exam ORDER BY exam_date DESC; GO
CREATE OR ALTER PROCEDURE Exam_GetById (@id INT) AS SELECT * FROM dbo.Exam WHERE exam_id=@id;

CREATE OR ALTER PROCEDURE Exam_Update (@id INT,@date DATE,@dur INT,@course INT) AS
BEGIN
    IF NOT EXISTS(SELECT 1 FROM dbo.Exam WHERE exam_id=@id) BEGIN RAISERROR('Exam not found',
    UPDATE dbo.Exam SET exam_date=@date, duration=@dur, course_id=@course WHERE exam_id=@id;
END;
GO

CREATE OR ALTER PROCEDURE Exam_Delete (@id INT) AS
BEGIN
    IF EXISTS(SELECT 1 FROM dbo.Student_Exam WHERE exam_id=@id) BEGIN RAISERROR('Cannot delet
    DELETE FROM dbo.Exam WHERE exam_id=@id;
END;
GO

CREATE OR ALTER PROCEDURE ExamQuestion_Insert (@e INT,@c INT,@q INT) AS
BEGIN
    IF NOT EXISTS(SELECT 1 FROM dbo.Exam WHERE exam_id=@e) BEGIN RAISERROR('Exam not found',1
    IF NOT EXISTS(SELECT 1 FROM dbo.Question WHERE course_id=@c AND question_id=@q) BEGIN RAI
    INSERT INTO dbo.Exam_Question(exam_id, course_id, question_id) VALUES(@e,@c,@q);
END;
GO

CREATE OR ALTER PROCEDURE ExamQuestion_Get (@e INT) AS
BEGIN
    SELECT eq.exam_id, eq.course_id, eq.question_id, q.type, q.description
    FROM dbo.Exam_Question eq
    JOIN dbo.Question q ON q.course_id = eq.course_id AND q.question_id = eq.question_id
    WHERE eq.exam_id = @e
    ORDER BY q.question_id;
END; GO

CREATE OR ALTER PROCEDURE ExamQuestion_Delete (@e INT,@c INT,@q INT) AS
BEGIN DELETE FROM dbo.Exam_Question WHERE exam_id=@e AND course_id=@c AND question_id=@q; END
```

CRUD Procedures (Junctions & StudentCourse / StudentExam)

```
CREATE OR ALTER PROCEDURE BranchTrack_Insert (@b INT,@t INT) AS
BEGIN
    IF NOT EXISTS(SELECT 1 FROM dbo.Branch WHERE branch_id=@b) BEGIN RAISERROR('Branch not fo
    IF NOT EXISTS(SELECT 1 FROM dbo.Track WHERE track_id=@t) BEGIN RAISERROR('Track not found
```

```
    INSERT INTO dbo.Branch_Track(branch_id,track_id) VALUES(@b,@t);
END; GO

CREATE OR ALTER PROCEDURE BranchTrack_Delete (@b INT,@t INT) AS
BEGIN DELETE FROM dbo.Branch_Track WHERE branch_id=@b AND track_id=@t; END; GO

CREATE OR ALTER PROCEDURE TrackCourse_Insert (@t INT,@c INT) AS
BEGIN
    IF NOT EXISTS(SELECT 1 FROM dbo.Track WHERE track_id=@t) BEGIN RAISERROR('Track not found
    IF NOT EXISTS(SELECT 1 FROM dbo.Course WHERE course_id=@c) BEGIN RAISERROR('Course not fo
    INSERT INTO dbo.Track_Course(track_id, course_id) VALUES(@t,@c);
END; GO

CREATE OR ALTER PROCEDURE TrackCourse_Delete (@t INT,@c INT) AS
BEGIN DELETE FROM dbo.Track_Course WHERE track_id=@t AND course_id=@c; END; GO

CREATE OR ALTER PROCEDURE InstructorCourse_Insert (@i INT,@c INT) AS
BEGIN
    IF NOT EXISTS(SELECT 1 FROM dbo.Instructor WHERE ins_id=@i) BEGIN RAISERROR('Instructor n
    IF NOT EXISTS(SELECT 1 FROM dbo.Course WHERE course_id=@c) BEGIN RAISERROR('Course not fo
    INSERT INTO dbo.Instructor_Course(ins_id, course_id) VALUES(@i,@c);
END; GO

CREATE OR ALTER PROCEDURE InstructorCourse_Delete (@i INT,@c INT) AS
BEGIN DELETE FROM dbo.Instructor_Course WHERE ins_id=@i AND course_id=@c; END; GO

CREATE OR ALTER PROCEDURE StudentCourse_Insert (@s INT,@c INT,@g INT)
AS
BEGIN
    IF NOT EXISTS(SELECT 1 FROM dbo.Student WHERE student_id=@s) BEGIN RAISERROR('Student not
    IF NOT EXISTS(SELECT 1 FROM dbo.Course WHERE course_id=@c) BEGIN RAISERROR('Course not fo
    IF EXISTS(SELECT 1 FROM dbo.Student_Course WHERE student_id=@s AND course_id=@c) BEGIN RA
    INSERT INTO dbo.Student_Course(student_id, course_id, grade) VALUES(@s,@c,@g);
END; GO

CREATE OR ALTER PROCEDURE StudentCourse_Update (@s INT,@c INT,@g INT) AS
BEGIN
    IF NOT EXISTS(SELECT 1 FROM dbo.Student_Course WHERE student_id=@s AND course_id=@c) BEGI
    UPDATE dbo.Student_Course SET grade=@g WHERE student_id=@s AND course_id=@c;
END; GO

CREATE OR ALTER PROCEDURE StudentCourse_Delete (@s INT,@c INT) AS
BEGIN DELETE FROM dbo.Student_Course WHERE student_id=@s AND course_id=@c; END; GO

CREATE OR ALTER PROCEDURE StudentExam_Insert (@s INT,@e INT)
AS
BEGIN
    IF NOT EXISTS(SELECT 1 FROM dbo.Student WHERE student_id=@s) BEGIN RAISERROR('Student not
    IF NOT EXISTS(SELECT 1 FROM dbo.Exam WHERE exam_id=@e) BEGIN RAISERROR('Exam not found',1
    IF EXISTS(SELECT 1 FROM dbo.Student_Exam WHERE student_id=@s AND exam_id=@e) BEGIN RAISER
    INSERT INTO dbo.Student_Exam(student_id, exam_id, total_grade) VALUES(@s,@e,NULL);
END; GO
```

```
CREATE OR ALTER PROCEDURE StudentExam_Get (@s INT,@e INT) AS SELECT * FROM dbo.Student_Exam W
CREATE OR ALTER PROCEDURE StudentExam_Delete (@s INT,@e INT) AS DELETE FROM dbo.Student_Exam

CREATE OR ALTER PROCEDURE StudentExamAnswer_Get (@s INT,@e INT) AS
BEGIN SELECT * FROM dbo.Student_Exam_Answer WHERE student_id=@s AND exam_id=@e ORDER BY quest
```

Advanced Procedures

Below are the crucial procedures that implement core domain logic: exam generation, answer submission, and correction. These are production-ready, contain validation and error handling, and include example usage.

> **Important design choice**: `ExamGeneration` inserts selected MCQ and TF rows into `Exam_Question`. Because schema lacks persistent order, clients must `ORDER BY NEWID()` at retrieval for randomized view or `ORDER BY q.question_id` for deterministic order. `ExamAnswer` expects answers as CSV of `choice_id` (numeric values) in the order produced by the client `SELECT` (i.e., client must fetch exam questions order and present answers in that sequence). Procedures enforce that the student is attached to the exam before accepting answers.

ExamGeneration

Purpose: Create an `Exam` for a course by randomly selecting N MCQ and M TF questions. Optionally use supplied date and duration.

Parameters:

- `@CourseName VARCHAR(50)` — name of course
- `@MCQ_Count INT` — number of MCQ questions to pick
- `@TF_Count INT` — number of True/False questions to pick
- `@ExamDate DATE` — date for the exam
- `@Duration INT` — duration in minutes
- `@NewExamID INT OUTPUT` — returns generated exam_id

Behavior & Validation:

- Validates Course exists.
- Ensures enough MCQ and TF questions exist.
- Generates a new `exam_id` safely using `TABLOCKX` to avoid race conditions on `MAX`.
- Inserts `Exam` and inserts selected `Exam_Question` rows.
- Final order is not persisted (no question order column). Clients must shuffle read order if they want randomized presentation.

SQL:

```
CREATE OR ALTER PROCEDURE ExamGeneration
    @CourseName    VARCHAR(50),
    @MCQ_Count     INT,
    @TF_Count      INT,
    @ExamDate      DATE,
    @Duration      INT,
    @NewExamID     INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    BEGIN TRAN;
    DECLARE @CourseID INT;

    -- Input validation
    IF @MCQ_Count < 0 OR @TF_Count < 0
    BEGIN ROLLBACK; RAISERROR('Question counts must be non-negative',16,1); RETURN; END
    IF @MCQ_Count = 0 AND @TF_Count = 0
    BEGIN ROLLBACK; RAISERROR('Select at least one question',16,1); RETURN; END
    IF @Duration <= 0 OR @ExamDate IS NULL
    BEGIN ROLLBACK; RAISERROR('Invalid date or duration',16,1); RETURN; END

    SELECT @CourseID = course_id FROM dbo.Course WHERE course_name = @CourseName;
    IF @CourseID IS NULL BEGIN ROLLBACK; RAISERROR('Course not found',16,1); RETURN; END

    IF (SELECT COUNT(*) FROM dbo.Question WHERE course_id = @CourseID AND type = 'MCQ') < @MC
       OR (SELECT COUNT(*) FROM dbo.Question WHERE course_id = @CourseID AND type = 'TF') < @
    BEGIN ROLLBACK; RAISERROR('Not enough questions',16,1); RETURN; END

    -- Safe Exam ID generation
    SELECT @NewExamID = ISNULL(MAX(exam_id), 5000) + 1 FROM dbo.Exam WITH (TABLOCKX);

    INSERT INTO dbo.Exam(exam_id, exam_date, duration, course_id)
    VALUES(@NewExamID, @ExamDate, @Duration, @CourseID);

    -- Select and insert MCQ
    INSERT INTO dbo.Exam_Question(exam_id, course_id, question_id)
    SELECT TOP (@MCQ_Count) @NewExamID, course_id, question_id
    FROM dbo.Question
    WHERE course_id = @CourseID AND type = 'MCQ'
    ORDER BY NEWID();

    -- Select and insert TF
    INSERT INTO dbo.Exam_Question(exam_id, course_id, question_id)
    SELECT TOP (@TF_Count) @NewExamID, course_id, question_id
    FROM dbo.Question
    WHERE course_id = @CourseID AND type = 'TF'
    ORDER BY NEWID();

    COMMIT;
END;
GO
```

Example:

```
DECLARE @examid INT;
EXEC ExamGeneration
    @CourseName='Full-Stack Web Development',
    @MCQ_Count=5,
    @TF_Count=3,
    @ExamDate='2026-02-01',
    @Duration=90,
    @NewExamID=@examid OUTPUT;
SELECT @examid AS ExamID;
```

Edge cases:

- Concurrency: `TABLOCKX` prevents simultaneous `MAX(exam_id)` collisions.
- Randomization: To view a randomized order, clients must run:

  ```
  SELECT q.*, NEWID() AS rnd
  FROM Exam_Question eq JOIN Question q ON eq.course_id=q.course_id AND eq.question_id=q.q
  WHERE eq.exam_id = @examid
  ORDER BY NEWID(); -- or by rnd
  ```

- If you require deterministic persistence of order, schema must be extended with an order column.

ExamAnswer

Purpose: Accept a student's answers for a given `exam_id`. `AnswersCSV` is a comma-separated list of `choice_id` values (numbers), in the same order as the client fetch of `Exam` questions.

Parameters:

- `@ExamID INT`
- `@StudentFName VARCHAR(50)`
- `@StudentLName VARCHAR(50)`
- `@AnswersCSV VARCHAR(MAX)` — CSV of `choice_id` values in question order (e.g., `'1,2,4,1,true'` — for this schema, use choice ids like `'1,2,3,4'`).

Behavior & Validation:

- Validates student exists.
- Validates exam exists.
- Validates the student is attached to the exam (`Student_Exam` exists for that student/exam). **Client or admin must perform StudentExam_Insert first** to register the student for that exam. This prevents students from answering exams they are not assigned to.

- Prevents duplicate attempt: if `Student_Exam.total_grade` is non-NULL (already graded), refuses. If `Student_Exam_Answer` records already exist for that student/exam, refuses (prevents multiple submissions).
- Parses `AnswersCSV` into a table preserving order.
- Fetches exam questions in deterministic order (ORDER BY `course_id, question_id`) — client must have followed same order when composing `AnswersCSV`.
- Validates answer count matches question count.
- Inserts student answers into `Student_Exam_Answer.stud_answer` as the `choice_id` string, and sets `is_correct` = NULL (will be set by `ExamCorrection`).
- Uses transaction and rolls back on validation failure.

SQL:

```
CREATE OR ALTER PROCEDURE ExamAnswer
    @ExamID INT,
    @StudentFName  VARCHAR(50),
    @StudentLName  VARCHAR(50),
    @AnswersCSV VARCHAR(MAX)
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;
    BEGIN TRAN;

    DECLARE @StudentID INT;

    -- Find student
    SELECT @StudentID = student_id FROM dbo.Student
    WHERE student_fname = @StudentFName AND student_lname = @StudentLName;

    IF @StudentID IS NULL
    BEGIN
        ROLLBACK; RAISERROR('Student not found',16,1); RETURN;
    END

    -- Verify exam exists
    IF NOT EXISTS (SELECT 1 FROM dbo.Exam WHERE exam_id = @ExamID)
    BEGIN
        ROLLBACK; RAISERROR('Exam not found',16,1); RETURN;
    END

    -- Verify student is attached to exam (Student_Exam record)
    IF NOT EXISTS (SELECT 1 FROM dbo.Student_Exam WHERE student_id = @StudentID AND exam_id =
    BEGIN
        ROLLBACK; RAISERROR('Student is not registered/attached to this exam',16,1); RETURN;
    END

    -- Prevent re-submission if answers already exist or graded
    IF EXISTS (SELECT 1 FROM dbo.Student_Exam_Answer WHERE student_id = @StudentID AND exam_i
    BEGIN
        ROLLBACK; RAISERROR('Student has already submitted answers for this exam',16,1); RETU
```

```
    END

    IF EXISTS (SELECT 1 FROM dbo.Student_Exam WHERE student_id = @StudentID AND exam_id = @Ex
    BEGIN
        ROLLBACK; RAISERROR('Exam already graded for this student',16,1); RETURN;
    END

    -- Parse answers CSV preserving order
    DECLARE @Answers TABLE (seq INT IDENTITY(1,1), choice_id_str VARCHAR(50));
    INSERT INTO @Answers (choice_id_str)
    SELECT LTRIM(RTRIM(value)) FROM STRING_SPLIT(@AnswersCSV, ',') ORDER BY (SELECT NULL);

    -- Get exam questions in deterministic order (must match client order)
    DECLARE @Questions TABLE (seq INT IDENTITY(1,1), course_id INT, question_id INT);
    INSERT INTO @Questions (course_id, question_id)
    SELECT eq.course_id, eq.question_id
    FROM dbo.Exam_Question eq
    WHERE eq.exam_id = @ExamID
    ORDER BY eq.course_id, eq.question_id;

    -- Validate counts match
    IF (SELECT COUNT(*) FROM @Answers) <> (SELECT COUNT(*) FROM @Questions)
    BEGIN
        ROLLBACK; RAISERROR('Number of answers does not match number of exam questions',16,1)
    END

    -- Insert Student_Exam_Answer rows
    INSERT INTO dbo.Student_Exam_Answer(student_id, exam_id, course_id, question_id, stud_ans
    SELECT
        @StudentID,
        @ExamID,
        q.course_id,
        q.question_id,
        a.choice_id_str,
        NULL
    FROM @Questions q
    JOIN @Answers a ON q.seq = a.seq;

    COMMIT;
END;
GO
```

Example:

1. Ensure Student_Exam exists:

```
EXEC StudentExam_Insert @s=4001, @e=5001;
```

2. Student answers:

```
EXEC ExamAnswer @ExamID=5001, @StudentFName='Mohamed', @StudentLName='Ali', @AnswersCSV='1,2,
```

Edge cases:

- Answer count mismatch → error.
- Student not registered to exam → error.
- Duplicate submissions → prevented.

ExamCorrection

Purpose: Grade a student's submission for a given `exam_id`. Compares `Student_Exam_Answer.stud_answer` (choice id string) to the correct `Choice.choice_id` (where `is_correct=1`) to mark `is_correct` and compute `Student_Exam.total_grade` as an integer percent.

Parameters:

- `@ExamID INT`
- `@StudentFName VARCHAR(50)`
- `@StudentLName VARCHAR(50)`

Behavior & Validation:

- Locates student id by name.
- Ensures there's at least one answer row for this student/exam.
- Updates `Student_Exam_Answer.is_correct` to 1 or 0 by matching student `stud_answer` to the `Choice.choice_id` (convert types as needed).
- Calculates total correct count and computes percent = `CAST((correct * 100.0) / total AS INT)`.
- Updates `Student_Exam.total_grade`.
- Returns a summary row with student_id, exam_id, correct_count, total_questions, final_percent.

SQL:

```
CREATE OR ALTER PROCEDURE ExamCorrection
    @ExamID INT,
    @StudentFName VARCHAR(50),
    @StudentLName VARCHAR(50)
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @StudentID INT, @Total INT, @Correct INT;

    SELECT @StudentID = student_id FROM dbo.Student WHERE student_fname = @StudentFName AND s
    IF @StudentID IS NULL BEGIN RAISERROR('Student not found',16,1); RETURN; END

    SELECT @Total = COUNT(*) FROM dbo.Student_Exam_Answer WHERE student_id = @StudentID AND e
    IF @Total = 0 BEGIN RAISERROR('No answers found to grade',16,1); RETURN; END

    -- Update correctness: compare stud_answer (choice_id stored as string) with the correct
    UPDATE sea
    SET is_correct = CASE WHEN TRY_CAST(sea.stud_answer AS INT) = c.choice_id THEN 1 ELSE 0 E
```

```
    FROM dbo.Student_Exam_Answer sea
    JOIN dbo.Choice c
      ON sea.course_id = c.course_id
     AND sea.question_id = c.question_id
     AND c.is_correct = 1
    WHERE sea.student_id = @StudentID
      AND sea.exam_id = @ExamID;

    SELECT @Correct = COUNT(*) FROM dbo.Student_Exam_Answer WHERE student_id = @StudentID AND

    UPDATE dbo.Student_Exam
    SET total_grade = CASE WHEN @Total > 0 THEN CAST((@Correct * 100.0) / @Total AS INT) ELSE
    WHERE student_id = @StudentID AND exam_id = @ExamID;

    SELECT
        @StudentID AS student_id,
        @ExamID AS exam_id,
        @Correct AS correct_count,
        @Total AS total_questions,
        CAST((@Correct * 100.0) / @Total AS INT) AS final_grade_percent;
END;
GO
```

Example:

```
EXEC ExamCorrection @ExamID=5001, @StudentFName='Mohamed', @StudentLName='Ali';
```

Edge cases:

- If `stud_answer` values are not numeric, `TRY_CAST` yields NULL and comparison fails (counts as incorrect).
- If a question has no `Choice.is_correct = 1` configured, update will treat it as incorrect (data issue — must fix question bank).

Reports & Supporting Procedures

Course_Topics

Purpose: Returns topics for a course.

```
CREATE OR ALTER PROCEDURE Course_Topics (@crs_id INT)
AS
BEGIN
    SELECT c.course_name, t.topic_name
    FROM dbo.Course c
    INNER JOIN dbo.Topic t ON c.course_id = t.course_id
    WHERE c.course_id = @crs_id;
```

```
END;
GO
```

Example:

```
EXEC Course_Topics @crs_id = 3001;
```

Exam_Questions

Purpose: Returns exam questions and choices for review (freeform report).

```
CREATE OR ALTER PROCEDURE Exam_Questions (@ex_id INT)
AS
BEGIN
    SELECT eq.exam_id, q.description, c.choice_id, c.choice_text, c.is_correct
    FROM dbo.Exam_Question eq
    INNER JOIN dbo.Question q
      ON eq.course_id = q.course_id AND eq.question_id = q.question_id
    INNER JOIN dbo.Choice c
      ON eq.course_id = c.course_id AND eq.question_id = c.question_id
    WHERE eq.exam_id = @ex_id
    ORDER BY q.question_id, c.choice_id;
END;
GO
```

Example:

```
EXEC Exam_Questions @ex_id = 5001;
```

GetStudentsByTrackId

Purpose: Get students assigned to a specific track and branch.

Signature: `@TrackId INT, @BranchId INT`

```
CREATE OR ALTER PROCEDURE GetStudentsByTrackId (@TrackId INT, @BranchId INT)
AS
BEGIN
    SELECT
        s.student_id, s.student_fname, s.student_lname, s.student_age, s.student_gender,
        b.branch_id, b.branch_name, t.track_name
    FROM dbo.Student s
    INNER JOIN dbo.Track t ON s.track_id = t.track_id
    INNER JOIN dbo.Branch_Track bt ON t.track_id = bt.track_id
    INNER JOIN dbo.Branch b ON bt.branch_id = b.branch_id
    WHERE s.track_id = @TrackId AND b.branch_id = @BranchId
    ORDER BY s.student_fname;
```

```
END;
GO
```

Example:

```
EXEC GetStudentsByTrackId @TrackId=106, @BranchId=6;
```

Instructor_Courses (Instructor workload)

Purpose: Returns list of courses an instructor teaches and student count per course.

```
CREATE OR ALTER PROCEDURE Instructor_Courses (@ins_id INT)
AS
BEGIN
    SELECT ic.course_id AS course_id,
           c.course_name,
           COUNT(DISTINCT sc.student_id) AS number_of_students
    FROM dbo.Instructor_Course ic
    INNER JOIN dbo.Course c ON ic.course_id = c.course_id
    LEFT JOIN dbo.Student_Course sc ON sc.course_id = ic.course_id
    WHERE ic.ins_id = @ins_id
    GROUP BY ic.course_id, c.course_name;
END;
GO
```

Example:

```
EXEC Instructor_Courses @ins_id = 9002;
```

Student_Exam_Model_Answer

Purpose: Report per student per exam: question description, student answer (choice text), correct answer.

Signature: `@exam_id INT, @student_id INT`

```
CREATE OR ALTER PROCEDURE Student_Exam_Model_Answer (@exam_id INT, @student_id INT)
AS
BEGIN
    SELECT q.description,
           sc.choice_text AS student_answer,
           c.choice_text AS correct_answer,
           sa.student_id, sa.exam_id
    FROM dbo.Student_Exam_Answer sa
    INNER JOIN dbo.Question q
        ON sa.course_id = q.course_id AND sa.question_id = q.question_id
    INNER JOIN dbo.Choice c
```

```
        ON q.course_id = c.course_id AND q.question_id = c.question_id AND c.is_correct = 1
    LEFT JOIN dbo.Choice sc
        ON sc.course_id = sa.course_id AND sc.question_id = sa.question_id
           AND sc.choice_id = TRY_CAST(sa.stud_answer AS INT)
    WHERE sa.exam_id = @exam_id AND sa.student_id = @student_id
    ORDER BY q.question_id;
END;
GO
```

Example:

```
EXEC Student_Exam_Model_Answer @exam_id=5001, @student_id=4001;
```

Student_grade

Purpose: Return student's enrolled courses and grades.

Signature: `@st_id INT`

```
CREATE OR ALTER PROCEDURE Student_grade (@st_id INT)
AS
BEGIN
    SELECT CONCAT(s.student_fname,' ', s.student_lname) AS Full_Name,
           c.course_name, sc.grade
    FROM dbo.Student s
    INNER JOIN dbo.Student_Course sc ON s.student_id = sc.student_id
    INNER JOIN dbo.Course c ON c.course_id = sc.course_id
    WHERE s.student_id = @st_id;
END;
GO
```

Example:

```
EXEC Student_grade @st_id = 4001;
```

Exam Workflow (End-to-End) — Detailed Steps & SQL Examples

1. **Populate course, topics, question bank and choices** using `Course_Insert`, `Topic_Insert`, `Question_Insert`, `Choice_Insert`.
2. **Enroll students** using `StudentCourse_Insert` (or bulk insert).
3. **Register students for an exam**:
    - Either create `Student_Exam` rows (manual) or have an automated process:

```
EXEC StudentExam_Insert @s=4001, @e=5001;
```

4. **Generate exam** (Instructor):

```
DECLARE @eid INT;
EXEC ExamGeneration @CourseName='Full-Stack Web Development', @MCQ_Count=5, @TF_Count=3,
```

5. **Student fetches exam questions** (client side must fetch in a stable order and record that order):

```
SELECT q.course_id, q.question_id, q.type, q.description,
       c.choice_id, c.choice_text
FROM dbo.Exam_Question eq
JOIN dbo.Question q ON eq.course_id = q.course_id AND eq.question_id = q.question_id
JOIN dbo.Choice c ON c.course_id = q.course_id AND c.question_id = q.question_id
WHERE eq.exam_id = @eid
ORDER BY NEWID(); -- randomize display to student
```

> Client must present questions in this new randomized order. When student submits answers, the client must send the answers in the same order as shown.

6. **Student submits answers**:

```
EXEC ExamAnswer @ExamID = @eid, @StudentFName='Mohamed', @StudentLName='Ali', @AnswersCS
```

7. **Correct answers and compute grade**:

```
EXEC ExamCorrection @ExamID=@eid, @StudentFName='Mohamed', @StudentLName='Ali';
```

8. **Retrieve result & model answers**:

```
EXEC Student_Exam_Model_Answer @exam_id=@eid, @student_id=4001;
SELECT * FROM dbo.Student_Exam WHERE student_id=4001 AND exam_id=@eid;
```

Reports & Queries (Examples & Expected Output)

Students by Track & Branch (SP: `GetStudentsByTrackId`)

SQL

```
EXEC GetStudentsByTrackId @TrackId=106, @BranchId=6;
```

Expected output columns: `student_id | student_fname | student_lname | student_age | student_gender | branch_name | track_name`

Student grades per course (SP: `Student_grade`)

```
EXEC Student_grade @st_id = 4001;
```

Expected output: rows with `Full_Name | course_name | grade`

Instructor teaching load (SP: `Instructor_Courses`)

```
EXEC Instructor_Courses @ins_id = 9002;
```

Expected output: `course_id | course_name | number_of_students`

Exam questions and choices (SP: `Exam_Questions`)

```
EXEC Exam_Questions @ex_id = 5001;
```

Expected output: `exam_id | description | choice_id | choice_text | is_correct`

Student answers vs model answers (SP: `Student_Exam_Model_Answer`)

```
EXEC Student_Exam_Model_Answer @exam_id = 5001, @student_id = 4001;
```

Expected output: For each question `description | student_answer | correct_answer | student_id | exam_id`

Course topics (SP: `Course_Topics`)

```
EXEC Course_Topics @crs_id = 3001;
```

Expected output: `course_name | topic_name`

Data Integrity & Transactions

Transactions usage:

- Procedures `ExamGeneration`, `ExamAnswer` use explicit transactions with `SET XACT_ABORT ON` to ensure atomic operations.
- `ExamGeneration` uses `TABLOCKX` to avoid race when computing `MAX(exam_id) + 1`.
- `ExamAnswer` and `ExamCorrection` wrap logic in transactions so partial writes do not occur.

Concurrency handling:

- For ID generation, `TABLOCKX` is used as a simple safe approach. For high concurrency, a sequence object (`CREATE SEQUENCE`) is preferred and recommended for future schema change.

- `ExamAnswer` prevents double submission by checking `Student_Exam_Answer` existence before insert and using transaction.

Failure modes:

- Procedures raise `RAISERROR` and rollback when validations fail.
- Data consistency is guaranteed if procedures are used; ad-hoc inserts should follow FK order.

Security & Safety

Why `Student_Exam_Answer` is read-only via SPs:

- To enforce validation, prevent tampering and ensure consistent auditing and grading. Application code should call `ExamAnswer`, not write directly.

Duplicate attempt prevention:

- `ExamAnswer` checks if the student already has `Student_Exam_Answer` rows or `Student_Exam.total_grade` not NULL and raises error.

Data validation strategy:

- Use stored procedures for all mutating operations.
- Validate inputs (non-negative counts, existing course/exam/student).
- Use `TRY_CAST` when converting user-submitted `stud_answer` values to integers.

Access control (recommendation):

- Create minimal roles:
  - `db_admin` — full DDL/DML.
  - `db_instructor` — `SELECT/EXECUTE/INSERT/UPDATE` on allowed objects and execute SPs.
  - `db_student` — `EXEC` only for read-only SPs and `ExamAnswer`.
- Example GRANTs:

```
CREATE ROLE db_instructor;
GRANT EXECUTE ON SCHEMA::dbo TO db_instructor;
GRANT SELECT, INSERT, UPDATE ON dbo.Student_Exam TO db_instructor;
-- Give more scoped grants as organization requires
```

Encryption & connections:

- Use TLS for DB connections. Enable Transparent Data Encryption (TDE) and backup encryption on production servers.

Performance Considerations

Indexes to add (suggestions):

```
-- Indexes for read-heavy operations
CREATE NONCLUSTERED INDEX IX_Exam_Question_ExamId ON dbo.Exam_Question (exam_id);
CREATE NONCLUSTERED INDEX IX_Question_Course_Type ON dbo.Question (course_id, type);
CREATE NONCLUSTERED INDEX IX_Student_Course_StudentId ON dbo.Student_Course (student_id);
CREATE NONCLUSTERED INDEX IX_StudentExam_ExamId ON dbo.Student_Exam (exam_id);
CREATE NONCLUSTERED INDEX IX_StudentExamAnswer_ExamId_Student ON dbo.Student_Exam_Answer (exa
```

Query tuning tips:

- Use proper joins with indexed columns.
- Avoid `SELECT *` in production queries.
- For large Question banks, consider partitioning by course or use filtered indexes on `type`.

Scaling notes:

- For heavy concurrency on exam generation, replace `MAX(exam_id)+1` pattern with `SEQUENCE` object:

```
CREATE SEQUENCE dbo.ExamSeq AS INT START WITH 5001;
```

- Consider caching frequently read reference tables (Course, Track) at application tier.

Testing & Validation

Test cases for core procedures (happy path + error case):

1. `ExamGeneration`

Happy path

```
DECLARE @eid INT;
EXEC ExamGeneration @CourseName='Full-Stack Web Development', @MCQ_Count=5, @TF_Count=3, @Exa
SELECT @eid;
-- Expectation: exam created with id returned; Exam_Question rows count = 8
```

Error case – insufficient questions

```
EXEC ExamGeneration @CourseName='Full-Stack Web Development', @MCQ_Count=500, @TF_Count=0, @E
-- Expectation: RAISERROR('Not enough questions')
```

2. `ExamAnswer`

Happy path

```
EXEC StudentExam_Insert @s=4001, @e=5001; -- ensure attached
EXEC ExamAnswer @ExamID=5001, @StudentFName='Mohamed', @StudentLName='Ali', @AnswersCSV='1,2,
-- Expectation: inserts Student_Exam_Answer rows (count=4)
```

Error case – student not registered

```
EXEC ExamAnswer @ExamID=5001, @StudentFName='Unregistered', @StudentLName='User', @AnswersCSV
-- Expectation: RAISERROR('Student not found') or 'Student is not registered/attached to this
```

3. `ExamCorrection`

Happy path

```
EXEC ExamCorrection @ExamID=5001, @StudentFName='Mohamed', @StudentLName='Ali';
-- Expectation: returns student_id, exam_id, correct_count, total_questions, final_grade_perc
```

Error case – no answers

```
EXEC ExamCorrection @ExamID=9999, @StudentFName='Noone', @StudentLName='Nobody';
-- Expectation: RAISERROR('No answers found to grade')
```

Automated tests: Use `tSQLt` (suggestion) to wrap the above as unit tests; CI should run scripts to drop/recreate DB in isolated environment and execute tSQLt test suite.

Deployment Notes

Order of execution (safe apply):

1. Create database and configure settings.
2. Create tables in order: `Branch` , `Track` , `Instructor` , `Branch_Track` , `Course` , `Track_Course` , `Instructor_Course` , `Topic` , `Student` , `Student_Course` , `Question` , `Choice` , `Exam` , `Exam_Question` , `Student_Exam` , `Student_Exam_Answer` .
3. Create indexes (nonclustered).
4. Create triggers (see next section).
5. Create stored procedures (in dependency order; CRUD first, then business SPs).
6. Run seed script.

Sample deployment script (skeleton):

```
-- 1. CREATE DATABASE ExaminationSystem
-- 2. USE ExaminationSystem
-- 3. run schema.sql (tables)
-- 4. run constraints/indexes/triggers
```

```
-- 5. run sprocs.sql
-- 6. run seed_data.sql
```

Backups:

```
-- Full backup
BACKUP DATABASE ExaminationSystem TO DISK = 'C:\Backups\ExaminationSystem_FULL.bak' WITH FORM

-- Restore
RESTORE DATABASE ExaminationSystem FROM DISK = 'C:\Backups\ExaminationSystem_FULL.bak' WITH R
```

Restore differential and log strategies described in operations runbook (not included to keep this doc focused on schema/SPs).

Triggers (Important ones & rationale)

Triggers implemented in the system (examples):

1. `trg_exam_question_course_match` — Prevents adding a question to an exam that belongs to a different course than the exam.
2. `trg_track_supervisor_same_track` — Ensures supervisor referenced in Track belongs to same track (business rule).
3. `trg_student_answer_invalid` — Prevents inserting answers for students not registered to exam.
4. `trg_answer_correctness` — Validates `is_correct` is 0 or 1.

Example: trg_exam_question_course_match

```
CREATE OR ALTER TRIGGER trg_exam_question_course_match
ON dbo.Exam_Question
FOR INSERT
AS
BEGIN
    IF EXISTS (
        SELECT 1
        FROM inserted i
        JOIN dbo.Exam e ON e.exam_id = i.exam_id
        WHERE e.course_id <> i.course_id
    )
    BEGIN
        RAISERROR('Question must belong to the same course as exam',16,1);
        ROLLBACK;
    END
END;
GO
```

Note: Triggers are safety nets; main validation should be in SPs.

Security & Access Control (Recommendation)

Roles & privileges:

- `DBA` — full rights (create/alter/drop).
- `InstructorRole` — `EXECUTE` on procedure schema, SELECT on Course/Question/Choice/Exam/Exam_Question/Student_Exam (as needed), INSERT on Student_Exam (to attach students), SELECT on Student for their students.
- `StudentRole` — `EXECUTE` only on `ExamAnswer`, `StudentExamAnswer_Get`, and `Student_grade` (READ own data).

Grant example:

```
CREATE ROLE db_instructor;
GRANT EXECUTE ON SCHEMA::dbo TO db_instructor;
GRANT SELECT ON dbo.Question TO db_instructor;
GRANT SELECT ON dbo.Choice TO db_instructor;
-- scope privileges further as desired
```

Auditing suggestions:

- Enable SQL Server Audit to log `EXEC` of `ExamAnswer`, `ExamCorrection`, and modifications to `Student_Exam_Answer`.
- Add `created_by`, `created_at` columns if auditing fine-grained change is required (schema change).

Troubleshooting & Common Errors

Common errors & fixes:

- `FK` insertion errors: Ensure insertion order respects FK constraints. Use seed script that inserts parents first.
- `Msg 8144: too many arguments specified` : Occurs when SP signature changed — drop procedure and recreate.
- `Students cannot submit` : Ensure `Student_Exam` exists for the student-exam pair prior to calling `ExamAnswer`.
- `Exam order appears MCQ then TF` : This is expected when not ordering at SELECT time. Use `ORDER BY NEWID()` or deterministic `ORDER BY` when fetching from `Exam_Question`.

Fix example for insertion conflict on Track supervisor:

- If `Track.supervisor_id` references an instructor that doesn't exist, insert instructor first or set `supervisor_id` to NULL then update.

Appendices & Useful Snippets

Useful SELECTs for verification

```
-- Counts
SELECT COUNT(*) AS BranchCount FROM dbo.Branch;
SELECT COUNT(*) AS TrackCount FROM dbo.Track;
SELECT COUNT(*) AS CourseCount FROM dbo.Course;
SELECT COUNT(*) AS QuestionCount FROM dbo.Question;
SELECT COUNT(*) AS ChoiceCount FROM dbo.Choice;
SELECT COUNT(*) AS ExamCount FROM dbo.Exam;
SELECT COUNT(*) AS StudentExamCount FROM dbo.Student_Exam;
SELECT COUNT(*) AS StudentExamAnswerCount FROM dbo.Student_Exam_Answer;

-- Find answers without student_exam parent
SELECT * FROM dbo.Student_Exam_Answer sea
LEFT JOIN dbo.Student_Exam se ON sea.student_id = se.student_id AND sea.exam_id = se.exam_id
WHERE se.student_id IS NULL;
```

Pagination snippet

```
SELECT * FROM dbo.Course ORDER BY course_name
OFFSET 0 ROWS FETCH NEXT 50 ROWS ONLY;
```

Use SEQUENCE for exam IDs (recommended future change)

```
CREATE SEQUENCE dbo.ExamSeq AS INT START WITH 5001 INCREMENT BY 1;
-- Then ExamGeneration can simply SELECT NEXT VALUE FOR dbo.ExamSeq
```

Glossary

- **Track:** A program or curriculum (e.g., "Data Management") that can be offered at multiple branches.
- **Branch:** Physical location where courses/tracks are offered.
- **Course:** A module or subject (e.g., "Full-Stack Web Development").
- **Topic:** Subtopics of a course (e.g., "HTML").
- **Question:** Item in the question bank, tied to a specific course. Types: MCQ, TF.
- **Choice:** Candidate answer for a question; `is_correct` indicates the correct choice.
- **Exam:** A collection of questions (Exam_Question) tied to a course and scheduled at a date/time.
- **Student_Exam:** Register a student to an exam; holds final computed grade.
- **Student_Exam_Answer:** The student's per-question answer for a specific exam attempt.
- **MCQ:** Multiple Choice Question.
- **TF:** True/False question.

- **Model Answer:** The correct `Choice.choice_id` for a question.

Conclusion

This documentation presents a full, production-ready relational schema for `ExaminationSystem`, covering the data model, business rules, stored procedures (CRUD and domain logic), reporting queries, transaction and concurrency considerations, deployment steps, security guidance, testing, and troubleshooting. The design uses normalized structures, composite keys for course-scoped questions, and stored procedures that encapsulate business logic and protect data integrity.

Why this design is correct & production-ready:

- Clear separation of responsibilities and normalization to 3NF.
- Stored procedures encapsulate validation, easing audit and consistent behavior.
- Triggers act as safety nets for invariants.
- Reports and procedures support the operational needs (generation, submission, and correction of exams).
- Recommendations for sequences, indexes, roles, and auditing provide a path to harden production operations.
